

Báo cáo Kỹ thuật: AI Dev Guardian

AI Engineering Governance Agent — Người gác cổng chất lượng code trước mỗi lần `git push`

Phiên bản: MVP · Ngày cập nhật: 30/07/2026

1. Bài toán: Tam giác thất bại của các công cụ Code Review hiện nay

Hiện tại, các engineering team phải đối mặt với "tam giác đánh đổi" không thể dung hòa từ 3 lớp bảo vệ truyền thống — nâng cấp một lớp không giải quyết được điểm yếu của 2 lớp còn lại:

1. Human Code Review (thủ công) — Phụ thuộc hoàn toàn vào băng thông và sự tập trung của reviewer. Khi team mở rộng hoặc nhịp độ release tăng, review thủ công trở thành "nút thắt cổ chai" (bottleneck) làm chậm tiến độ, hoặc bị làm qua loa dẫn đến sót lỗi nghiêm trọng.

2. Static Analysis / Linter truyền thống (Deterministic) — Bắt lỗi cú pháp rất tốt nhưng hoàn toàn "mù" trước context và logic nghiệp vụ riêng của từng dự án. Bên cạnh đó, việc cấu hình luật rất phức tạp (phải học các ngôn ngữ DSL/Rego hoặc viết file YAML rườm rà).

3. AI Reviewer thế hệ đầu (Post-push / Pull Request Bots) — mắc cùng lúc 3 lỗi cấu trúc:

- Sai vị trí (Wrong Gate)** — Hầu hết chạy ở tầng CI/CD hoặc PR Bot trên GitHub/GitLab. Dev phải chờ push code lên mới nhận phản hồi, gây ngắt quãng luồng làm việc (context switching) và tốn chi phí CI/Compute không cần thiết.
- Thảm họa Hallucination** — Ảo giác của LLM khiến nó tự tin cảnh báo sai (false positive) — ví dụ: đếm sai số dòng code, bắt lỗi không tồn tại. Hệ quả là dev bị kiệt sức vì cảnh báo rác (alert fatigue) và dần bỏ qua mọi gợi ý của AI.
- Rủi ro Auto-fix âm thầm** — Nhiều công cụ cố tình tự động sửa code (auto-patch). Một bản patch sai do AI sinh ra nếu lọt qua review sẽ âm thầm đưa bug nghiêm

trọng vào sản phẩm.

Đây chính là khoảng trống mà AI Dev Guardian được thiết kế để lấp đầy: chặn đúng chỗ (trước push, không phải sau), hiểu đúng ngữ cảnh (Policy-as-Code, không phải DSL cứng), và không tự ý sửa code thay con người.

2. Giải pháp

AI Dev Guardian là một agent quản trị kỹ thuật (Engineering Governance Agent) chạy như một **pre-push gate**: kiểm tra diff *trước khi* code rời máy dev, không phải dọn dẹp *sau khi* đã lên remote.

Nguyên lý thiết kế cốt lõi:

- **Deterministic + Probabilistic kết hợp** — luật kiểm tra chắc chắn (secret lộ, circular dependency, static analysis rule) chạy bằng công cụ xác định 100%; chỉ những gì thực sự cần *hiểu ngữ nghĩa* mới giao cho LLM.
- **Policy-as-Code** — luật không phải cấu hình cứng từ nhà cung cấp, mà là file Markdown do chính team viết (`.guardian/policies/*.md`), LLM đọc và áp dụng trực tiếp.
- **Không tự động vá code** — mọi vi phạm chỉ đi kèm một `promptToFix` sẵn sàng copy-paste vào AI assistant của chính dev — quyết định sửa thế nào vẫn luôn thuộc về con người.
- **Một verdict duy nhất** — `PASS` hoặc `BLOCK`, không có vùng xám.

So sánh với các công cụ hiện có

Tiêu chí	Human Review	Static Analysis (ESLint, SonarQube...)	AI PR Bot thể hệ đầu (CodeRabbit, Copilot Review...)	AI Dev Guardian
Thời điểm chạy	Sau khi mở PR, phụ thuộc lịch reviewer	IDE/CI	Sau khi push — tầng CI/CD hoặc PR bot	Trước khi push — git hook cục bộ

Hiểu logic nghiệp vụ riêng	Có, nhưng cảm tính, không nhất quán	Không — chỉ bất cứ pháp/style	Có phần, nhưng không cho định nghĩa Policy-as-Code riêng	Có — Policy-as-Code bằng Markdown
Cấu hình luật	Không cấu hình được	Phức tạp — DSL/Rego hoặc YAML rườm rà	Thường giới hạn qua UI/prompt cố định	1 file Markdown + YAML frontmatter
Chống Hallucination	N/A (con người vẫn có thể sai/sót)	N/A (deterministic)	Thường 1 lượt LLM, không công khai cơ chế xác minh	5 lớp : grounding, CoT, self-consistency, AI-as-Judge, RAG-lite
Auto-fix code	Không	Có (fix) cho rule cứng, an toàn	Một số tự động đề xuất/commit fix — rủi ro patch sai	Không bao giờ — chỉ sinh promptToFix
Chi phí vận hành	Thời gian reviewer (khó định lượng)	Miễn phí	Thuê bao theo seat, ~10-20 USD/dev/tháng	Theo token thực tế + cache — ~\$3-4/dev/tháng
Nút thắt khi team scale	Có — bottleneck rõ rệt	Không	Không, nhưng cộng dồn chi phí CI/compute	Không — chạy song song, cache tái sử dụng

Đặc điểm của các công cụ bên thứ ba trong bảng phản ánh xu hướng chung của nhóm sản phẩm, không phải benchmark đo trực tiếp trên phiên bản mới nhất của từng công cụ.

3. Tính năng & Công nghệ lõi

Nhóm	Tính năng	Công nghệ lõi
Kiểm tra xác định	Secret Scan	Regex trên các dòng diff mới thêm (+)

Kiểm tra xác định	Architecture Check	<code>madge</code> — phát hiện <code>circular dependency</code> , chỉ báo cáo cycle liên quan tới diff
Kiểm tra xác định	Semgrep Check (tùy chọn)	Binary <code>semgrep</code> , ruleset <code>p/security-audit</code> , lọc theo dòng diff thật đã thêm
Kiểm tra ngữ nghĩa	LLM Policy Check	Anthropic Claude / OpenAI GPT, 1 lần gọi/file, chỉ kèm policy đúng scope
Ngữ cảnh	RAG-lite	Đọc thêm tối đa 3 file vệ tinh được import trực tiếp — TS/JS, Python, C/C++, Go
Hiệu năng	Diff-hash caching	SHA-256, LRU 20 hash <code>PASS</code> gần nhất, lưu tại <code>.git/guardian_cache.json</code>
An toàn	Prompt-as-a-Fix	Không tự sinh code vá lỗi — chỉ sinh prompt nhờ AI khác sửa, 1 template chuẩn dùng chung
Vận hành	Interactive Git Hook	<code>guardian install-hook</code> , hỏi <code>Y/n</code> khi có TTY, fail-open khi chạy trong CI

5 lớp khiên chống ảo giác LLM

- **Grounding theo schema** — `policyId` bị ràng buộc bằng `enum` đúng danh sách policy đã nạp cho file đó; model không thể trích dẫn một luật không tồn tại.
- **Reasoning-first** — field `reasoning` khai báo *đầu tiên* trong schema, bắt buộc model suy luận trước khi kết luận.
- **Grounded Evidence** — `evidenceSnippet` phải là dòng trích dẫn **y hệt** trong diff thật; dòng comment-only bị loại khỏi tập bằng chứng hợp lệ.
- **Self-consistency** — mọi phát hiện `critical` phải được xác nhận lại ở lượt gọi thứ 2 với cùng prompt; không khớp `policyId` cả 2 lần → bị loại.
- **AI-as-a-Judge** — mọi vi phạm sống sót được một model thứ 2 tự đếm lại/suy luận lại từ code thật, độc lập với model đầu tiên.

4. Kiến trúc hệ thống đã triển khai

Luồng xử lý từ git push hoặc guardian check qua CLI, core engine, các checker song

song, verdict aggregator, đến reporter và kết quả cuối cùng, với đường bypass cache riêng ở lề phải. Trigger git push / guardian check --staged CLI / entrypoint CLI entrypoint Git diff reader Core engine / orchestrator Policy router and cache Định tuyến + cache PASS Checkers layer — chạy song song Secret scan Leaked keys Architecture Layer rules Semgrep Pattern rules LLM + judge Policy check Verdict aggregator Cache hit → bỏ qua LLM Reporter Fix-prompt generator Terminal renderer Exit code + khung màu kết quả

Sơ đồ trên mô tả đúng những gì **đã được cài đặt và chạy được**, chia thành 4 tầng rõ ràng:

- **CLI / Entrypoint** — CLI entrypoint (đọc lệnh guardian check) và Git diff reader (lấy diff từ staged area hoặc từ pre-push hook).
- **Core Engine / Orchestrator** — Policy router and cache (định tuyến policy theo scope + kiểm tra diff-hash cache) và Verdict aggregator (gộp kết quả, tính PASS/BLOCK).
- **Checkers layer** (lồng bên trong Orchestrator, chạy song song) — Secret scan, Architecture, Semgrep, LLM + judge.
- **Reporter** — Fix-prompt generator (build promptToFix theo template dùng chung) và Terminal renderer (khung màu theo mức độ nghiêm trọng).

Mọi khối trong sơ đồ tương ứng 1:1 với module thật trong src/: src/cli.ts, src/orchestrator.ts, src/checks/*.ts, src/report/*.ts — đây là ảnh chụp đúng trạng thái code hiện tại, không phải sơ đồ khái niệm.

5. Cách hoạt động dựa trên sơ đồ kiến trúc

Theo đúng thứ tự trong sơ đồ:

1. **Trigger** — dev chạy git push (đã cài guardian install-hook) hoặc gọi tay guardian check --staged.
2. **CLI / Entrypoint** — CLI đọc lệnh, lấy diff tương ứng.
3. **Policy router and cache** — loại bỏ file test/fixture, định tuyến policy có scope khớp file thay đổi, tính SHA-256 hash của diff để tra cache.
4. **Fan-out sang Checkers layer** — 4 checker chạy đồng thời (Promise.all):

Secret scan, Architecture, Semgrep luôn chạy; riêng LLM + judge bị **bỏ qua nếu hash trùng một lần PASS gần đây** — đường bypass cache vẽ riêng ở lề phải sơ đồ.

- 5. **Fan-in vào Verdict aggregator** — kết quả 4 nhánh gộp lại một chỗ; verdict là BLOCK nếu có ít nhất 1 vi phạm mức medium trở lên, ngược lại là PASS (và ghi hash vào cache).
- 6. **Reporter** — Fix-prompt generator build promptToFix cho từng vi phạm, sau đó Terminal renderer in khung màu kết quả cuối cùng cho dev.

6. Lợi ích hiệu năng

- **Chạy song song, không tuần tự** — tổng thời gian check giới hạn bởi checker chậm nhất (thường là LLM call), không phải tổng cộng cả 4.
- **Cache-bypass tức thì** — diff không đổi thì bỏ qua hoàn toàn bước LLM (phần chậm và tốn phí nhất); 3 checker miễn phí vẫn luôn chạy nên độ an toàn không đổi.
- **Không trả tiền 2 lần cho cùng 1 diff** — LRU 20 hash PASS gần nhất, sống sót qua việc chuyển branch qua lại.
- **BLOCK không bao giờ được cache** — đảm bảo mọi bản sửa lỗi luôn được kiểm tra lại thật.
- **Fail-open trên mọi thành phần tùy chọn** — thiếu API key, thiếu binary semgrep, hoặc lỗi mạng ở judge pass đều chỉ log cảnh báo và bỏ qua đúng bước đó, không bao giờ crash hay tự ý BLOCK một push hợp lệ.

7. Lộ trình tương lai

Đã hoàn thành

Tính năng	Trạng thái
Secret scan (regex, deterministic)	✓ Done
LLM policy check (Claude/GPT, Policy-as-Code)	✓ Done
Grounding evidence + policyId enum	✓ Done

Chain-of-thought reasoning field	✓ Done
AST comment/string annotation	✓ Done
Self-consistency cho critical	✓ Done
AI-as-a-Judge (2nd pass)	✓ Done
Prompt-as-a-Fix (template dùng chung, tiếng Anh)	✓ Done
Diff-hash caching (SHA-256, LRU 20)	✓ Done
RAG-lite (TS/JS, Python, C/C++, Go)	✓ Done
Circular dependency detection (madge)	✓ Done
Semgrep integration (tùy chọn)	✓ Done
Interactive pre-push git hook	✓ Done

Đang chờ / Kế hoạch

Tính năng	Mô tả
CI / GitHub Action gate	Chạy Guardian trực tiếp trên PR, comment kết quả tự động
Architecture Rules policy category	Luật hướng phụ thuộc giữa layer, kiểm tra xác định, không cần LLM
Git Workflow policy category	Luật đặt tên branch, format commit message
Testing Standards policy category	Luật yêu cầu file test tương ứng khi thêm code mới
Dependency Rules policy category	Chặn dependency mới không nằm trong danh sách được duyệt

Business Requirements policy category	Gắn thay đổi code với yêu cầu sản phẩm (cơ chế cụ thể còn TBD)
Jira integration	Tự động gắn violation vào ticket theo dõi
Component ownership qua git blame	Gắn <code>promptToFix</code> với đúng người đã viết dòng code vi phạm
Policy-driven severity cho circular dependency	Chuyển severity hiện đang <code>hardcode medium</code> sang cấu hình qua policy

8. Chi phí vận hành API — chứng minh tính khả thi

Guardian mặc định ưu tiên Anthropic khi có cả 2 API key (`resolveLLMClient` trong `src/checks/llm/resolveClient.ts`): model chính là `claude-sonnet-5`, model cho lượt judge là `claude-haiku-4-5` (rẻ hơn, dùng cho việc xác minh độc lập chứ không cần suy luận đầy đủ).

Bảng giá (Anthropic, giá chuẩn — chưa tính giá ưu đãi ra mắt của Sonnet 5 đang áp dụng tới 31/08/2026 là \$2/\$10 mỗi triệu token, rẻ hơn ~33% so với số dưới đây):

Model	Input / 1M token	Output / 1M token	Dùng cho
Claude Sonnet 5	\$3.00	\$15.00	LLM Policy Check (lượt chính + self-consistency)
Claude Haiku 4.5	\$1.00	\$5.00	AI Judge (lượt xác minh độc lập)

Giả định để ước tính (nêu rõ để minh bạch, không phải số đo thực tế):

- Team 5 dev, mỗi dev trung bình 4 lần push/ngày, mỗi push có 3 file cần LLM check (đã qua `routePolicies` lọc).
- Mỗi lượt check: ~4.000 input token (system prompt + policy + nội dung file + RAG satellite + diff), ~400 output token.
- Cache diff-hash giúp bỏ qua ~30% lượt check (diff trùng lần `PASS` gần đây).
- Judge pass chỉ chạy khi có vi phạm sống sót sau grounding — ước tính ~25% số

lượt check.

- Self-consistency lượt 2 chỉ chạy khi phát hiện **critical** ở lượt đầu — ước tính hiếm, ~5%.

Tính chi phí mỗi lượt check (USD):

Thành phần	Input	Output	Chi phí	Tần suất	Chi phí kỳ vọng
Main pass (Sonnet 5)	4.000 tok → \$0.012	400 tok → \$0.006	\$0.018	100 %	\$0.0180
Self-consistency (Sonnet 5)	như trên	như trên	\$0.018	5%	\$0.0009
Judge pass (Haiku 4.5)	2.500 tok → \$0.0025	150 tok → \$0.00075	\$0.0032 5	25%	\$0.0008
Tổng mỗi lượt check					≈ \$0.0197 (~\$0.02)

Quy ra theo tháng (22 ngày làm việc, đã trừ ~30% nhờ cache):

Quy mô team	Lượt check/tháng (sau cache)	Chi phí/tháng
1 dev	~185	≈ \$3.7
5 dev	~924	≈ \$18.5
20 dev	~3.696	≈ \$74
50 dev	~9.240	≈ \$185

Kết luận khả thi: với team 5 người, Guardian tốn khoảng **\$18.5/tháng** — rẻ hơn phí thuê bao 1 seat GitHub Copilot Business (~\$19/tháng) cho **cả team**, và không đáng kể so với chi phí thực tế của một sự cố duy nhất mà Guardian ngăn được (một secret bị lộ production, hoặc một circular dependency gây bug khó debug hàng giờ). Chi phí còn được kiểm soát chủ động bởi chính kiến trúc: 3/4 check luôn miễn phí (deterministic), cache bỏ qua lượt LLM khi diff không đổi, và judge pass chỉ chạy khi thật sự có vi phạm

cần xác minh.

Tài liệu tổng hợp từ README.md và trạng thái code thật của dự án tại thời điểm 30/07/2026.